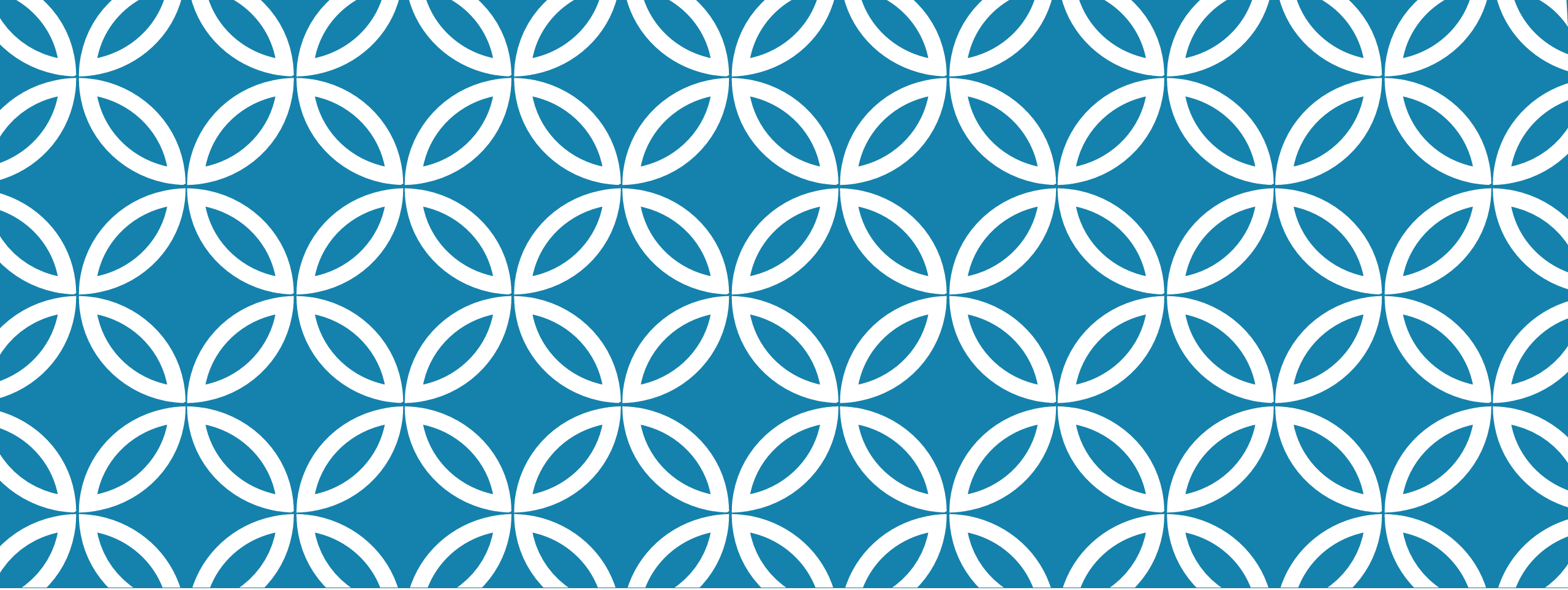


NESNE TABANLI PROGRAMLAMA-I

Öğr. Gör. Erhan AKAGÜNDÜZ

KAPSÜLLEME NEDİR?

- Kapsülleme, bir sınıfın içindeki değişkenlerin (variable) ve metotların dışarıdan erişimini kısıtlamaktır.
- Bu yöntem, yazılım geliştirme sürecinde veri güvenliğini sağlamak için kullanılır.
- Örneğin, bir öğrencinin sınav notlarına herkesin müdahale etmesi doğru olmaz.
- Kapsülleme sayesinde verileri sadece izin verdiğimiz şekilde okutabilir veya değiştirebiliriz.
- Python'da kapsülleme genellikle __ (çift alt çizgi) kullanılarak yapılır.
- Bu yöntem, nesne yönelimli programlamada en önemli kavramlardan biridir.

NEDEN KAPSÜLLEMEYE İHTİYAÇ DUYARIZ?

- Gerçek hayatta bazı bilgilere herkesin ulaşmasını istemeyiz.
- Örneğin, bankacılık sisteminde müşterinin şifreleri direkt erişime açık olmamalıdır.
- Aynı şekilde öğrenci notları veya maaş bilgileri de korunmalıdır.
- Eğer değişkenler korunmazsa, yanlışlıkla değiştirilmesi sistem hatalarına yol açabilir.
- Kapsülleme bu riskleri ortadan kaldırır ve güvenli kod yazmamızı sağlar.
- Ayrıca, yazılım ekiplerinde kodun kontrolsüz değiştirilmesini de engeller.

KAPSÜLLEME OLMADAN BİR SINIF

- Bu örnekte exam1 ve exam2 değişkenleri açık şekilde tanımlandı.
- Dışarıdan bu değişkenlere rahatlıkla erişebiliyoruz.
- Hatta değerlerini istediğimiz gibi değiştirebiliyoruz.
- Bu durum güvenlik açığına yol açar.
- Yani kapsülleme yapılmadığında veriler korunamaz.

```
class RegisterCourse:
    def __init__(self):
        self.name = "Ahmet"
        self.surname = "Kaleli"
        self.exam1 = 74
        self.exam2 = 80

register = RegisterCourse()
print(register.exam1)
print(register.exam2)
```

AÇIK DEĞİŞKENLERDEKİ SORUN

- Görüldüğü gibi öğrencinin sınav notlarını doğrudan değiştirdik.
- Bu aslında uygulamada çok ciddi bir problemdir.
- Çünkü yanlışlıkla veya kötü niyetle değerler değiştirilebilir.
- İşte tam da bu noktada kapsülleme devreye girer.
- Verilerin güvenliğini sağlamak için özel (private) değişkenler kullanırız.

```
register.exam1 = 50
register.exam2 = 60
print(register.exam1)
print(register.exam2)
```

Çıktı:

```
50
60
```

PRİVATE DEĞİŞKEN KULLANIMI

- Değişkenlerin önüne __ ekleyerek onları private hale getirdik.
- Artık exam1 ve exam2 sadece bu sınıf içinde erişilebilir.
- Dışarıdan erişmeye çalıştığımızda hata alırız.
- Böylece sınav notları korunmuş olur.
- Bu yöntem Python'da kapsüllemenin temel yoludur.

```
class RegisterCourse:
    def __init__(self):
        self.name = "Ahmet"
        self.surname = "Kaleli"
        self.__exam1 = 74
        self.__exam2 = 80
```

PRİVATE DEĞİŞKENLERE ERİŞİM HATASI

- Python hata vererek private değişkene doğrudan erişimi engelledi.
- Bu durum kapsüllemenin doğru çalıştığını gösterir.
- Artık dışarıdan not bilgisi okunamaz veya değiştirilemez.
- Bu sayede güvenlik sağlanmış olur.
- Ancak bazen bu bilgilere kontrollü şekilde erişmemiz gerekir.

```
register = RegisterCourse()
print(register.__exam1)
```

Çıktı:

```
AttributeError: 'RegisterCourse' object has no attribute '__exam1'
```

METOTLARDA KAPSÜLLEME

- Private sadece değişkenler için değil metotlar için de kullanılabilir.
- Burada __add() metodu dışarıdan çağrılamaz.
- Yani kurs ekleme işlemini sınıf dışında kullanamayız.
- Bu da metodun yanlış kullanılmasını engeller.
- Böylece sınıfın işleyişi korunur.

```
class RegisterCourse:
    def __init__(self):
        self.__courses = []
    def __add(self, course):
        self.__courses.append(course)
```

GETTER YÖNTEMİ

- Private değişkenlere doğrudan erişemeyiz.
- Bunun için getter adını verdiğimiz özel metotlar kullanırız.
- Getter sadece veriyi okumamıza izin verir.
- Dışarıdan veri değiştirilemez, sadece görüntülenir.
- Bu, kapsüllemenin en yaygın kullanım şeklidir.

```
class RegisterCourse:
    def __init__(self):
        self.__exam1 = 74
    def getExam1(self):
        return self.__exam1
```

GETTER KULLANIMI

- Burada getter fonksiyonunu kullanarak exam1 değerini okuduk.
- Private değişkene doğrudan erişmedik.
- Ama kontrollü bir şekilde değerini alabildik.
- Bu sayede hem güvenlik sağlandı hem de ihtiyaç karşılandı.
- Getter metotları büyük yazılımlarda oldukça önemlidir.

```
register = RegisterCourse()
print(register.getExam1())
```

Çıktı:

```
74
```

SETTER YÖNTEMİ

- Private değişkenlerin değerini değiştirmek için setter kullanırız.
- Setter, dışarıdan gelen veriyi kontrol ederek atama yapar.
- Bu kontrol sayesinde hatalı veya yanlış değerlerin atanması engellenir.
- Doğrudan erişimde böyle bir kontrol mümkün değildir.
- Setter kapsüllemenin ikinci temel ayağıdır.

```
class RegisterCourse:
    def __init__(self):
        self.__exam1 = 74
    def setExam1(self, newVal):
        self.__exam1 = newVal
```

SETTER KULLANIMI

- İlk başta sınav notu 74’tü.
- Setter fonksiyonu ile değerini 40 yaptık.
- Getter kullanarak tekrar okuduğumuzda değiştiğini gördük.
- Bu işlem kontrollü bir değişim sağlar.
- Doğrudan erişimle yapılan değişikliklerden çok daha güvenlidir.

```
register = RegisterCourse()
print(register.getExam1())
register.setExam1(40)
print(register.getExam1())
```

Çıktı:

```
74
40
```

SETTER İLE VALİDASYON

- Setter içinde koşullar (validasyonlar) ekleyebiliriz.
- Örneğin, sınav notunun 0'dan küçük veya 100'den büyük olmaması gerekir.
- Eğer böyle bir değer girilirse hata fırlatırız.
- Bu şekilde veriler her zaman doğru aralıkta kalır.
- Kapsülleme ile güvenlik ve veri doğruluğu sağlanmış olur.

```
def setExam1(self, newVal):
    if newVal < 0 or newVal > 100:
        raise ValueError("Sınav notu 0-100 aralığında olmalı")
    self.__exam1 = newVal
```

HATALI GİRDİ ÖRNEĞİ

- Burada yanlış bir sınav notu girmeye çalıştık.
- Setter fonksiyonu buna izin vermedi ve hata verdi.
- Bu, kapsüllemenin en önemli avantajıdır.
- Yanlış veri programın içine girmeden engellenir.
- Böylece yazılımın bütünlüğü korunur.

```
register.setExam1(-10)
```

Çıktı:

```
ValueError: Sınav notu 0-100 aralığında olmalı
```

GETTER VE SETTER'IN ÖNEMİ

- Getter olmadan private değişkenler okunamaz.
- Setter olmadan private değişkenler değiştirilemez.
- Bu iki yapı birlikte kapsüllemeyi tamamlar.
- Dışarıdan erişim engellenir ama kontrollü kullanım sağlanır.
- Yazılım geliştirme sürecinde güvenli veri yönetimi için çok önemlidir.
- Büyük projelerde kapsülleme olmazsa olmaz bir yöntemdir.

GENEL DEĞERLENDİRME

- Python’da kapsülleme, __ ile private tanımlama sayesinde yapılır.
- Bu sayede değişkenlere ve metotlara doğrudan erişim engellenir.
- Getter ve Setter yöntemleriyle kontrollü erişim sağlanır.
- Ayrıca setter içine validasyon ekleyerek veri doğruluğu korunabilir.
- Kapsülleme hem güvenlik hem de düzenli kod yazma açısından kritik öneme sahiptir.
- Yazılım dünyasında sürdürülebilir projeler için kapsülleme mutlaka kullanılmalıdır.